DHBW Ravensburg

# Intelligence Engineering

## Solving Real World Problems At Scale

Prof. Dr.-Ing. Mark Schutera | October 18, 2026 · *drifting stream*

LECTURE I

# Overview of a Machine Learning System

# This lecture in one slide

**I Overview of Machine Learning Systems**

The small part and the big picture

Understanding Machine Learning Systems

A **model** is only a small part from the system around it, and some might argue there are a lot of moments when not to reach for machine learning at all.

1

# The small part and the big picture

## Don’t confuse the model with the machine learning system.

# Enabling the delivery of the logic

The **model** is the small part of a machine learning system. With foundation models and AI engineering, that part got smaller still. The essential work of a machine learning system remains and stacks around business understanding, infrastructure, design pattern and data engineering.

Before learning about solving a problem with machine learning, let's first assess when it is actually a good idea to do so.

# (1) Learn

The problem you are solving, requires your system to have the capability to **learn**. Learning can look different depending on the problem and its context and boundary conditions. You already know about supervised learning, unsupervised learning and reinforcement learning paradigms.

Machine learning is an approach to (1) learn (2) complex patterns from (3) existing data and use these patterns to make (4) predictions on (5) unseen data.

# (2) Pattern

Learning requires the existence of a **pattern**, as opposed to noise or randomness. It is not trivial nor obvious whether there is a pattern in the data, and whether it is learnable. Complex pattern is more difficult to embed or state explicitly, this is where machine learning has an edge over classical approaches.

Machine learning is an approach to (1) learn (2) complex patterns from (3) existing data and use these patterns to make (4) predictions on (5) unseen data.

# (3) Data

Data is available, accessible or acquirable. The data is of **sufficient** quality and quantity to support learning, (and to validate for that matter). If your approach does not require data, it is because you build on embeddings or foundation models, which themselves are built on data in the first place, or you need to rely on data acquisition and curation as part of your system.

Machine learning is an approach to (1) learn (2) complex patterns from (3) existing data and use these patterns to make (4) predictions on (5) unseen data.

# (4) Predictions

Machine learning is about making predictions, **always**. Predictions can be of different types, such as classification, regression, ranking, clustering, generation and on and on. By reframing problems in terms of predictions, we can leverage machine learning to solve them. Predictions are approximations by nature and therefore come with uncertainty, which needs to be accounted for.

Machine learning is an approach to (1) learn (2) complex patterns from (3) existing data and use these patterns to make (4) predictions on (5) unseen data.

# (5) Unseen Data

In a closed system or domain, the data is finite and the system can be fully specified by optimization. However, often the domain is open and infinite, and the system needs to be able to **generalize** to unseen data. Of course, the unseen data is not completely unknown, it is drawn from the same distribution (i.i.d.) as the training data and is thus able to generalize to the unseen data inside a specific domain or distribution.

Machine learning is an approach to (1) learn (2) complex patterns from (3) existing data and use these patterns to make (4) predictions on (5) unseen data.

# Catalysts

A catalyst is a substance that increases the rate of reaction without undergoing change itself.

**The task is repetitive.**

In a sense that we have a lot of data to learn a pattern from.

**It scales.**

The need to scale a specific prediction capability, not only a few times.

**The context changes.**

Meaning that hard coded or other solutions will be brittle and break.

**Being wrong is not too bad.**

Approximations on unseen data. Failures need to be manageable.

# When Not to Use Machine Learning

**It is unethical.**

Well, that might also be a general rule. But doing stuff at scale, without human oversight, has its implications.*

**There's a simple solution.**

A simple solution, heuristic, or baseline should always be the first step regardless.

**It's not economically viable.**

The value of the prediction, needs to be greater than the cost of the system, including development, deployment and maintenance.

* More? Enroll to my Philosophy of AI course.

Describe two use-cases and the rationale for whether or not to use machine learning.

# One Machine Learning Use-Case and One which is not.

# Mapping the value, 2018

**The map: value in the trillions**

McKinsey sized **400+** use cases across **19** industries and nine business functions: deep learning promised **$3.5 to $5.8 trillion** in value per year, the largest shares in marketing and sales plus supply chain and manufacturing.

**The ground: months per model**

Algorithmia surveyed **745** practitioners in October 2019: most companies needed **8 to 90 days** to deploy a single model, 18% needed longer than 90 days, and deployment time grew year over year.

The value was mapped in trillions while shipping one model took a quarter: feasibility, not value, was the bottleneck.

[1, 2]

# Value over technical feasibility

SWE-bench Verified went from 60% to near 100% in a year. The frontier between hard today and works today **moves right** every quarter.

- **Deploy now** coding agents, recommender systems, fraud detection, machine translation
- **Invest** autonomous driving, medical diagnosis, agentic workflows end to end
- **Automate cheaply** speech transcription, document OCR
- **Skip** bespoke one off models

Eisenhower: value over technical feasibility

# The field is moving, 2026

**Adoption is near universal**

88% of organizations use AI in at least one function, 72% use gen AI. Gen AI reached 53% of the population in three years, faster than the PC or the internet.

**Value capture is not**

80% of users report personal productivity gains. Only **37%** of organizations can attribute any EBIT impact to AI, flat since 2025, and 6% qualify as high performers.

Adoption climbs, value capture flat

[3, 4]

# OpenAI measures the work

GDPval takes **1,320** real deliverables from **44** occupations across nine sectors, and has experts averaging 14 years in the field grade them head to head against the model.

Frontier models finish them roughly **100**× faster and **100**× cheaper. What is left is judgment, not speed.

OpenAI, GDPval, 2025 [5]. A benchmark built out of what people are actually paid to produce.

GDPval: 1320 tasks, 44 occupations

# Anthropic measures the traffic

The Economic Index reads Claude's own live traffic rather than a benchmark. **49%** of occupations show Claude on at least a quarter of their tasks.

Augmentation leads automation **52% to 45%**, and the single most common task is modifying software to correct errors, 6% of all usage.



Anthropic Economic Index, 2026 [6]. Broad across occupations, thin within them, and the densest use is code.

Economic Index: where Claude is used

2

# Understanding Machine Learning Systems

## Research and Production are not the same.

# Horizontal versus Vertical

Nothing motivates system thinking more than the need to go horizontal. Academia is vertical, with a narrow focus on a single problem, where drilling deep, fast is rewarded and focus allows to teach a wide range of concepts.

| | Research | Production |
|---|---|---|
| **Objectives** | State of the art, highly engineered | Stakeholder outcome driven |
| **Compute** | Training and high throughput | Fast inference with low latency |
| **Data** | Often static benchmark oriented | Shifting domains and structures |
| **Usability metrics** | Usually not considered | UI, Fairness, Robust, Interpretability |

# Stakeholders and Objectives

In research the objective is one number on one benchmark. Ship the same model as a product and five people want five different things from it: **conflicting requirements** are the norm, and not all of them are must haves.



ML engineers



Platform team



Product team



Sales team



Manager

Tinder for horses

Different stakeholders have different objectives and think in different solution spaces.

# Define the next step for horse tinder and line out how to reach that.

# Five people, five solution spaces

Each of them is right. The **objective** is what they are measured on, the solution space is where they will look for it.

| | Objective | Solution space |
|---|---|---|
| ML engineers | better matches | more data, a bigger model |
| Platform team | nothing pages at 3am | freeze the model, build the platform |
| Product team | the swipe feels instant | a latency budget of 100 ms |
| Sales team | revenue per match | rank the premium stallions first |
| Manager | margin | fewer models, and maybe fewer engineers |

# Leaderboards and Benchmarks

- **System** A leaderboard scores a **model**, and a system is not a model.
- **Metric** One metric on one frozen test set. Whether it measures what it claims is rarely argued.
- **Overfit** Every run against a benchmark stales it. GLUE was saturated within a year.
- **Chance** Scanning 10,000 seeds moves CIFAR 10 by 1.82 points, nothing else changed.

GLUE 2018. Trust me bro, nine tasks

[7–10]

# Computational priorities

Research optimises **throughput**, production optimises **latency**, and one buys the other.

| Processing | Latency | Throughput |
|---|---|---|
| One query at a time | 10 ms | 100 queries/s |
| Batches of 10 | 20 ms | 500 queries/s |
| Batches of 100 | 50 ms | 2,000 queries/s |

Every row is the same model on the same hardware. Only the batch changed.

Batching waits for the queue to fill, so each single request waits longer while the machine as a whole serves far more of them.

Training leans throughput: nobody is waiting on one gradient step. Serving leans latency: somebody is.

Batch size is not the only lever. Model size, sample size and how many models sit in one request all move both numbers, and they do not move them in the same direction.

[11]

# Latency is a distribution

Ten requests, latency in ms:

9.5, 9.7, 9.9, 10.0, 10.0, 10.0, 10.2, 10.4, 11.0, **90.0**

The mean is 18.1 ms. Nine of the ten requests were served in about 10 ms and not one of them took 18. The average describes a request that never happened.

Users get less patient.

2017, Akamai: a 100 ms delay can cost 7% of conversions.

2019, Booking.com: about 30% more latency cost about 0.5% in conversion, “a relevant cost for our business”.

2016, Google: over half of mobile users leave a page that takes more than three seconds.

[12–14]

# Percentiles, not averages

The same ten requests. The **median** is 10.0 ms, p90 is 11.0 ms, p99 is 90.0 ms.

$$p_q \;=\; \inf\Big\{\, t \;:\; F_\ell(t) \;\geq\; \tfrac{q}{100} \,\Big\}$$

A percentile is a promise: $q$ percent of requests finish in $p_q$ or less. The mean hides the outlier, the percentile is the outlier.

What you actually work on is the gap between two percentiles, the slowest tenth that is not the worst hundredth:

$$S_{90,99} \;=\; \{\, r \;:\; p_{90} \leq \ell(r) < p_{99} \,\}$$

That set is big enough to hold a repeatable cause and small enough to count. $S_{99,100}$ is usually one broken machine.

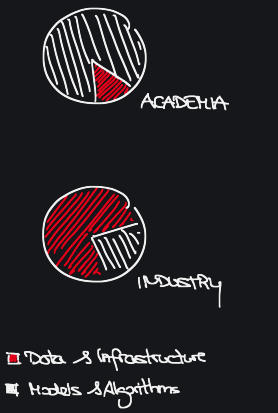
# Data in research

A research dataset is a teaching artefact. Somebody already did the work.

- Clean and well formatted
- Static, so the community can benchmark against it
- Quirks of the dataset are known and documented
- Open source scripts to feed your models
- Historical, already stored somewhere

Titanic [15]: 891 rows, 12 columns, and every one of its quirks is written up in a tutorial somewhere.

Karpathy: at Tesla you lose sleep over data




[16]

# Data in production

Nobody did the work. The data is a by-product of a business that was not run for your benefit.

- Messy, noisy, possibly unstructured
- **Likely biased, and you do not know how**
- Labels sparse, imbalanced, or incorrect
- Changing requirements can mean relabeling
- Privacy and regulatory concerns
- Generated by users, systems, third parties

# Perpetuating Bias

Machine learning algorithms do not predict the future, they encode the past, and then they act on it.

**The invisible hand** of performative prediction at scale: the prediction is not a reading of the world, it is a push on it.

2014 to 2018, Amazon built a résumé ranker on ten years of its own hiring. It learned to penalise the word “women’s” and graduates of two women’s colleges. The project was scrapped.

2015, Google Photos labelled two Black users as gorillas. The fix shipped, and three years later, was still to delete the label from the vocabulary.

[17–20]

# Interpretability

Research rewards performance, not interpretability. Industry asks for it anyway, from two directions:

- **Users**, business leaders and end users alike, to see why a decision was made, so they can trust it and catch what is wrong with it.
- **Developers**, to debug and improve the model at all.

Most of us use a fridge without understanding it. Fewer feel that way about the thing deciding on their mortgage. Only 19% of large companies are working on explainability.

Press 2: hand Claude recursive permission

# Human on the loop

Putting a human on the loop is the standard answer to an uninterpretable model. The evidence that it works is **thin**.

Supervising reliable automation degrades exactly the capability being supervised: vigilance falls, response to the first failure is poor, and the takeover you designed for arrives too late.

Victor et al., 2018: drivers supervising automation drove into a conflict object with **eyes on the threat** and **hands on the wheel**. Looking at it was not the same as expecting it.

So a human on the loop is a design decision that has to be tested, not a control you get for free by writing it into the spec.

[21]

# Why this is different from traditional software engineering

Traditional software engineering tests and versions **code**. A machine learning system has to test and version code, **data**, and everything the two produce together.

- How do you version a dataset, and diff two versions of one?
- How do you deploy into an environment that keeps moving?
- How do you monitor and debug a system with no stack trace?
- How do you avoid being harmful at scale?

Two books worth owning, and neither is about machine learning.

**Software Engineering at Google** [22]: engineering is programming integrated over time. Every question on the left is a question about time.

**The Mythical Man-Month** [23]: essential complexity against accidental. Data is essential complexity, and there is no silver bullet for it either.

**A trained model is the small box in the middle.**

You might argue that the engineering and development side of machine learning is more exciting.

# Most jobs will be, and already are in productionizing machine learning systems.

# References


1 Michael Chui et al. *Notes from the AI Frontier: Insights from Hundreds of Use Cases*. Discussion paper. McKinsey Global Institute, 2018.

2 Algorithmia. *2020 State of Enterprise Machine Learning*. Tech. rep. Algorithmia, Inc., 2019.

3 AI Index Steering Committee. *The AI Index 2026 Annual Report*. Tech. rep. Institute for Human-Centered AI, Stanford University, 2026.

4 McKinsey & Company. *The State of AI: Global Survey 2026*. Tech. rep. McKinsey & Company, QuantumBlack, 2026.

5 OpenAI. *GDPval: Evaluating AI Model Performance on Real-World Economically Valuable Tasks*. arXiv:2510.04374. 2025.

6 Anthropic. *Anthropic Economic Index Report: Economic Primitives*. Anthropic research report, January 2026. 2026.

7 Alex Wang et al. "GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding". In: *Proceedings of the 2018 EMNLP Workshop BlackboxNLP*. 2018, pp. 353–355.

8 Inioluwa Deborah Raji et al. "AI and the Everything in the Whole Wide World Benchmark". In: *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*. 2021.

# References


9 Timo Freiesleben and Sebastian Zezulka. *The Benchmarking Epistemology: Construct Validity for Evaluating Machine Learning Models*. arXiv:2510.23191. 2025.

10 David Picard. *torch.manual_seed(3407) is All You Need: On the Influence of Random Seeds in Deep Learning Architectures for Computer Vision*. arXiv:2109.08203. 2021.

11 Chip Huyen. *Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications*. Sebastopol, CA: O'Reilly Media, 2022.

12 Akamai Technologies. *State of Online Retail Performance, Spring 2017*. Tech. rep. Akamai Technologies, 2017.

13 Lukas Vermeer. *Moving Fast: Latency and Conversion at Booking.com*. Booking.com engineering. 2019.

14 Google. *The Need for Mobile Speed: How Mobile Latency Impacts Publisher Revenue*. Tech. rep. Google and DoubleClick, 2016.

15 Kaggle. *Titanic: Machine Learning from Disaster*. Kaggle getting started competition. 2012.

16 Andrej Karpathy. *Building the Software 2.0 Stack*. Train AI 2018, San Francisco. 2018.

17 Jeffrey Dastin. *Amazon Scraps Secret AI Recruiting Tool That Showed Bias Against Women*. Reuters, 10 October 2018. 2018.

# References


18 Tom Simonite. *When It Comes to Gorillas, Google Photos Remains Blind*. Wired, 11 January 2018. 2018.

19 Moritz Hardt. *The Invisible Hand of Prediction*. Simons Institute for the Theory of Computing, Berkeley. 2022.

20 Juan C. Perdomo et al. “Performative Prediction”. In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. PMLR. 2020, pp. 7599–7609.

21 Trent W. Victor et al. “Automation Expectation Mismatch: Incorrect Prediction Despite Eyes on Threat and Hands on Wheel”. In: *Human Factors* 60.8 (2018), pp. 1095–1116.

22 Titus Winters, Tom Manshreck, and Hyrum Wright. *Software Engineering at Google: Lessons Learned from Programming Over Time*. Sebastopol, CA: O’Reilly Media, 2020.

23 Frederick P. Brooks. *The Mythical Man-Month: Essays on Software Engineering*. Reading, MA: Addison-Wesley, 1975.